HUDM 5001 Programming for Data Science
Fall 2026 Teachers College, Columbia University

Mock Midterm Exam
ANSWER KEY

Practice for the in-class midterm on Tuesday, October 27
This mock covers the sessions up to SQLite. The real midterm covers everything we have learned before it.

About this mock test. This mock test is for you to get familiar with the upcoming midterm exam. No submission is needed for this mock test. The midterm will be similar to this test in overall design, but the number of questions may differ.

Name: _____ **UNI:** _____

Suggested time: 60–80 minutes.

Rules. To get the most out of this practice, try it under the same rules as the real exam:

1. Closed book. You may use **one** one-sided, letter-size sheet of notes (your "cheat sheet") and a calculator.
2. No computers, phones, tablets, smartwatches, earbuds, or AI tools. Put them away before you start.
3. You will **not** write any code. Every question asks you to read, trace, explain, or choose.
4. Part A: choose one answer for each question and mark it on the answer grid on page 2. Only the grid is scored.
5. Part B: write your answers in the space provided. You may cite line numbers (for example, "line 12") instead of copying code. Show your reasoning, because partial credit is given.
6. Unless a question says otherwise, assume that `import json, os, sqlite3`, `import numpy as np` and `import pandas as pd` have already been run, that each snippet runs by itself in a fresh Python 3 session, and that no warning messages appear.

Part	Points	Score
A. Multiple choice (25 questions × 2 points)	50	
B1. Reading comprehension: academic research	25	
Total	**75**	

Turn the page when you are ready to start.

Designed by Chenguang Pan (cp3280@tc.columbia.edu), refined by Claude.ai.

Part A: Answer grid

Mark **exactly one** letter for each question. Only this grid is scored for Part A. If you change an answer, erase the old mark completely.

1.	A	B	**C**	D
2.	**A**	B	C	D
3.	A	B	C	**D**
4.	A	**B**	C	D
5.	A	B	**C**	D
6.	A	**B**	C	D
7.	**A**	B	C	D
8.	A	B	C	**D**
9.	A	**B**	C	D
10.	A	B	**C**	D
11.	**A**	B	C	D
12.	A	B	C	**D**
13.	A	B	**C**	D
14.	**A**	B	C	D
15.	A	**B**	C	D
16.	A	B	C	**D**
17.	**A**	B	C	D
18.	A	B	**C**	D
19.	A	**B**	C	D
20.	A	B	C	**D**
21.	**A**	B	C	D
22.	A	**B**	C	D
23.	A	B	**C**	D
24.	A	B	C	**D**
25.	A	**B**	C	D

Part A: Multiple choice

25 questions × 2 points = 50 points. Choose the single best answer for each question and mark it on the answer grid (page 2).

1. What does this code print?

```
x = 5
y = x
x = 10
print(x, y)
```

A. `5 5`
B. `5 10`
✓ **C.** `10 5`
D. `10 10`

Key: C. `y = x` copies the value 5 into `y` at that moment. The later `x = 10` changes only `x`, so the output is `10 5`. *Tempting wrong answer:* D treats `y` as a permanent link to `x`. A Python variable is not a spreadsheet formula.

2. What does this code print?

```
print(18 / 4, 18 // 4, 18 % 4)
```

✓ **A.** `4.5 4 2`
B. `4.5 5 2`
C. `4 4.5 2`
D. `4.5 4 0.5`

Key: A. `/` is true division, so you get 4.5. `//` rounds *down* to 4. `%` is the remainder after taking out whole fours: $18 = 4 \cdot 4 + 2$. *Tempting wrong answer:* B assumes `//` rounds to the nearest whole number. It always rounds down.

3. The Series `temps` is built below. Which expression returns the temperature for Wednesday (65) **by its position**?

```
temps = pd.Series([68, 71, 65, 74],
                  index=["Mon", "Tue", "Wed", "Thu"])
```

A. `temps.loc[2]`
B. `temps.iloc[3]`
C. `temps.iloc["Wed"]`
✓ **D.** `temps.iloc[2]`

Key: D. `.iloc` uses integer positions that start at 0. Wednesday is the third entry, so its position is 2. *Tempting wrong answer:* B counts from 1, but `temps.iloc[3]` is Thursday (74). A fails with a `KeyError` because `.loc` expects a label. C fails with a `TypeError` because `.iloc` does not accept a label.

4. `students` has 5 rows and 3 columns. What does this code print?

```
students = pd.DataFrame({
    "name":  ["Maya", "Liam", "Noah", "Ava", "Zoe"],
    "major": ["Stats", "Learning", "Measurement", "Stats", "Learning"],
    "gpa":   [3.8, 3.5, 3.9, 3.6, 3.7],
})
print(students[["name", "gpa"]].shape, students["gpa"].shape)
```

A. `(2, 5) (5,)`
✓ **B.** `(5, 2) (5,)`
C. `(5, 2) (5, 1)`
D. `(2, 5) (1, 5)`

Key: B. Double brackets give a DataFrame with the 5 rows and the 2 chosen columns, so the shape is `(5, 2)`. A single column is a Series, which has only one dimension: `(5,)`. *Tempting wrong answer:* C gives the Series a second dimension. Only a DataFrame has columns.

5. What does this code print?

```
word = "data"
word.replace("a", "o")
print(word)
```

A. `doto`
B. `dota`
✓ **C.** `data`
D. `TypeError`

Key: C. Strings cannot be changed in place. `.replace()` returns a *new* string, and line 2 throws it away, so `word` is still `data`. *Tempting wrong answer:* A assumes `.replace()` edits `word` directly. (B is wrong as well, because `.replace()` replaces every match.)

6. What does this code print?

```
grades = {"Ann": 90, "Bob": 85}
print("Bob" in grades, 90 in grades, 85 in grades.values())
```

A. `True True True`
✓ **B.** `True False True`
C. `True False False`
D. `False True True`

Key: B. `in` on a dictionary checks the *keys*. `"Bob" in grades` is True, and `90 in grades` is False because 90 is a value, not a key. To check the values you need `grades.values()`, so `85 in grades.values()` is True. *Tempting wrong answer:* A treats `in` as a search through both keys and values.

7. What does this code print?

```
s = "regression"
print(s[2:5], s[-3:])
```

✓ **A.** `gre ion`
B. `gres ion`
C. `egr ion`
D. `gre regress`

Key: A. Positions start at 0 and the stop is left out, so `s[2:5]` takes positions 2, 3 and 4: `gre`. `s[-3:]` starts three characters from the end: `ion`. *Tempting wrong answer:* B includes the stop position. (D mixes up `s[-3:]` with `s[:-3]`.)

8. What does this code print?

```
df = pd.DataFrame({"team": ["A", "B", "A", "B", "A"],
                   "pts":  [10, 7, 12, 9, 8]})
means = df.groupby("team")["pts"].mean()
print(means["A"], means["B"])
```

A. `9.2 9.2`
B. `30 16`
C. `3 2`
✓ **D.** `10.0 8.0`

Key: D. `groupby("team")["pts"].mean()` gives one mean per team. Team A is $(10+12+8)/3 = 10.0$ and team B is $(7+9)/2 = 8.0$. *Tempting wrong answer:* B shows the group *sums* (30 and 16). The method here is `.mean()`, not `.sum()`.

9. Each line below is run by itself. Which one raises a `TypeError`?

A. `f"Total: {90}"`
✓ **B.** `"Total: " + 90`
C. `"Total: " * 2`
D. `"Total: " + str(90)`

Key: B. `+` cannot join a `str` and an `int`. You would first convert the number with `str(90)` (D) or use an f-string (A). *Tempting wrong answer:* C looks wrong but is legal: `str * int` repeats the text and gives `Total: Total: `.

10. What happens when this code runs?

```
df = pd.DataFrame({"a": [1, 2], "b": [3, 4]})
df.rename(columns={"a": "alpha"})
print(df["alpha"])
```

A. It prints the column's values, because `rename` changed `df` in place.
B. `KeyError`, because `rename` can only rename row labels, not columns.
✓ **C.** **`KeyError`, because `rename` returned a new DataFrame that was never assigned, so `df` still has the column `a`.**
D. `TypeError`, because `rename` needs a list of new names, not a dict.

Key: C. Table methods such as `.rename()` return a new table. Line 2 builds one and throws it away, so `df` is unchanged and `df["alpha"]` raises `KeyError: 'alpha'`. (The fix is `df = df.rename(...)`.) *Tempting wrong answer:* A expects the change to happen in place.

11. What does this code print?

```
scores = pd.Series([80, 90, np.nan, 70])
print(scores.mean(), scores.fillna(0).mean())
```

✓ **A.** `80.0 60.0`
B. `60.0 80.0`
C. `nan 60.0`
D. `60.0 60.0`

Key: A. pandas `.mean()` skips the missing value: $(80+90+70)/3 = 80.0$. After `fillna(0)` the missing score counts as a zero: $240/4 = 60.0$. *Tempting wrong answer:* C is how NumPy behaves. pandas skips missing values by default.

12. In the code below, `n` counts the rows with `pts` above 8. Which expression **always** gives the same number as `n`, for any DataFrame `df` that has a numeric column `pts`?

```
df = pd.DataFrame({"pts": [10, 7, 12, 9, 8]})
n = df[df["pts"] > 8].shape[0]
```

A. `df["pts"].sum() > 8`
B. `df["pts"] > 8`
C. `len(df["pts"] > 8)`
✓ **D.** `(df["pts"] > 8).sum()`

Key: D. The comparison gives one True or False per row, and `.sum()` counts the True values. That is exactly the number of rows the filter keeps. *Tempting wrong answer:* C: `len` counts every entry of the mask, True and False alike (here 5, not 3).

13. After these lines run, which statement is true?

```
a = [1, 2, 3]
b = a
c = [1, 2, 3]
b.append(4)
```

Tip. Lists, sets, dictionaries, NumPy arrays, and pandas Series and DataFrames are all *mutable* objects. Writing `b = a` does not copy anything. It only gives the same object a second name. So when you change the object through `b` (for example with `b.append(...)`), the change also shows up through `a`. To get a separate copy, write `b = a.copy()`. Also remember that `x is y` asks whether two names refer to the very same object, while `x == y` only asks whether their values are equal.

A. `a` is still `[1, 2, 3]`, because `b = a` made a separate copy.
B. `a == c` is `True`, because both lists hold the same values.
✓ **C. `a is b` is `True` and `a == c` is `False`.**
D. `a is c` is `True`, because both lists were built from the same numbers.

Key: C. `b = a` does not copy anything. `a` and `b` are two names for one list, so `b.append(4)` changes `a` too, and `a is b` is True. `c` is a separate list that still holds `[1, 2, 3]`, so `a == c` is now False. *Tempting wrong answer:* A believes that `b = a` copies the list.

14. What does this code print?

```
df = pd.DataFrame({"x": [10, 20, 30, 40, 50]})
print(df.loc[1:3]["x"].sum(), df.iloc[1:3]["x"].sum())
```

✓ **A.** `90 50`
B. `50 90`
C. `50 50`
D. `90 90`

Key: A. `.loc[1:3]` works with labels and *includes* the end label, so rows 1, 2 and 3 give $20+30+40 = 90$. `.iloc[1:3]` works with positions and *excludes* the end, so rows 1 and 2 give $20+30 = 50$. *Tempting wrong answer:* C treats both as if the end were left out.

15. What does this code print?

```
x = np.array([[2, 3, 1],
              [3, 1, 2]])
print(x[x > 1].sum(), (x > 1).sum())
```

A. `4 10`
✓ **B.** `10 4`
C. `4 4`
D. `12 4`

Key: B. `x > 1` is a True/False array. `x[x > 1]` keeps the elements where it is True: 2, 3, 3, 2, which add up to 10. `(x > 1).sum()` counts the True values, which is 4. *Tempting wrong answer:* C uses the mask itself as the values.

16. What happens when this code runs?

```
df = pd.DataFrame({"age":    [19, 25, 31, 22],
                   "member": [True, False, True, True]})
print(df[df["age"] > 20 and df["member"]])
```

A. It prints the rows where `age` is over 20 and `member` is `True`.
B. `KeyError`, because `member` is not a valid column name.
C. `TypeError`, because a number cannot be compared with a Boolean column.
✓ **D. `ValueError`, because `and` cannot combine two Series; `&` with parentheses is needed.**

Key: D. `and` needs a single True or False on each side, but a Series holds many of them, so pandas raises a `ValueError` ("The truth value of a Series is ambiguous"). The fix uses `&` and parentheses:

```
df[(df["age"] > 20) & df["member"]]
```

Tempting wrong answer: A reads `and` as plain English.

17. What does this code print?

```
with open("log.txt", "w") as f:
    f.write("A")
with open("log.txt", "a") as f:
    f.write("B")
with open("log.txt", "w") as f:
    f.write("C")
with open("log.txt", "a") as f:
    f.write("D")
with open("log.txt", "r") as f:
    print(f.read())
```

Tip. Mode `"w"` (write) first erases everything that is already in the file, and then writes the new text. Mode `"a"` (append) keeps what is already in the file and adds the new text at the end. Either mode creates the file if it does not exist yet.

✓ **A.** `CD`
B. `ABCD`
C. `ABD`
D. `D`

Key: A. Mode `"w"` erases the file before writing, and `"a"` adds to the end. So the file holds A, then AB, then only C (the second `"w"` wiped it), and finally CD. *Tempting wrong answer:* B treats `"w"` as if it appended.

18. What does this code print?

```
orders = pd.DataFrame({"cid": [1, 2, 3, 4, 5],
                       "amt": [50, 20, 70, 40, 30]})
cust = pd.DataFrame({"cid":  [3, 4, 5, 6],
                     "city": ["NYC", "LA", "NYC", "SF"]})
merged = pd.merge(orders, cust, on="cid", how="outer")
print(merged.shape)
```

A. `(3, 3)`
B. `(5, 3)`
✓ **C.** `(6, 3)`
D. `(9, 3)`

Key: C. An outer join keeps every key from both tables. The keys 1, 2, 3, 4, 5 and 6 make 6 rows, with the 3 columns `cid`, `amt` and `city`. *Tempting wrong answer:* B is the row count of a left join. The outer join also keeps `cid` 6 from the right table. (An inner join would give 3.)

19. A SQLite table `scores` is shown below (`NULL` means missing), and the cursor `cur` is connected to its database. What does the code print?

```
student | score
--------+------
a       | 80
b       | NULL
c       | 90
d       | NULL
e       | 70
```

```
query = "SELECT COUNT(*), COUNT(score), AVG(score) FROM scores"
print(cur.execute(query).fetchall()[0])
```

A. `(5, 5, 80.0)`
✓ **B.** `(5, 3, 80.0)`
C. `(3, 3, 80.0)`
D. `(5, 3, 48.0)`

Key: B. `COUNT(*)` counts rows (5). `COUNT(score)` counts only the scores that are not NULL (3). `AVG` ignores NULLs too: $(80+90+70)/3 = 80.0$. In pandas, `.count()` behaves like `COUNT(score)`. *Tempting wrong answer:* D treats each NULL as a 0 in the average ($240/5 = 48.0$).

20. What does this code print?

```
df = pd.DataFrame({"name":  ["Ann", "Bob", "Cara", "Dan"],
                   "score": [70, 95, 60, 85]})
top = df.sort_values("score", ascending=False)
print(top.iloc[0]["name"], top.loc[0]["name"], top.head(2)["score"].sum())
```

A. `Ann Ann 165`
B. `Cara Ann 130`
C. `Bob Bob 180`
✓ **D.** `Bob Ann 180`

Key: D. After sorting, Bob (95) is the first row, so `.iloc[0]` is Bob. But `sort_values` keeps each row's original label, and label 0 still belongs to Ann, so `.loc[0]` is Ann. The top two rows are Bob (95) and Dan (85), which add up to 180. *Tempting wrong answer:* C assumes the labels are renumbered after sorting.

21. A SQLite table `visits` is shown below, and the cursor `cur` is connected to its database. What does the code print?

```
page | device  | secs
-----+---------+-----
home | mobile  | 30
home | desktop | 95
shop | mobile  | 60
shop | mobile  | 100
blog | mobile  | 20
blog | desktop | 40
home | mobile  | 50
shop | desktop | 10
```

```
query = """
SELECT page, COUNT(*) AS n, ROUND(AVG(secs), 1) AS avg_secs
FROM visits
WHERE device = 'mobile'
GROUP BY page
ORDER BY avg_secs DESC
LIMIT 1
"""
print(cur.execute(query).fetchall()[0])
```

✓ **A.** `('shop', 2, 80.0)`
B. `('home', 3, 58.3)`
C. `('blog', 1, 20.0)`
D. `('shop', 2, 160)`

Key: A. `WHERE` keeps the five mobile visits first (home 30 and 50, shop 60 and 100, blog 20). `GROUP BY page` then averages them: home 40.0, shop 80.0, blog 20.0. `ORDER BY avg_secs DESC` puts shop first, `LIMIT 1` keeps only that row, and shop has `n` = 2. *Tempting wrong answer:* B ignores the `WHERE` filter (over all three home visits the average is 58.3).

22. What does this code print?

```
sales = pd.DataFrame({"store":  ["N", "S", "N", "S", "N", "E"],
                      "amount": [120, 80, 100, 90, 140, 60]})
avg = sales.groupby("store")["amount"].mean().to_dict()
avg["W"] = 0
gap = avg["N"] - avg["S"]
print(len(avg), gap)
```

A. `3 35.0`
✓ **B.** `4 35.0`
C. `4 190`
D. `4 1`

Key: B. The group means are E 60.0, N 120.0 and S 85.0. Assigning to the new key `"W"` adds a fourth entry, so `len(avg)` is 4. The gap is $120.0 - 85.0 = 35.0$. *Tempting wrong answer:* A forgets that assigning to a new key adds an entry. (C uses sums, and D uses counts.)

23. What does this code print?

```
df = pd.DataFrame({"id":    [1, 2, 3, 4, 5],
                   "score": [72, 58, 91, 65, 85]})
summary = {"n_pass": df[df["score"] >= 65].shape[0]}
with open("out.json", "w") as f:
    json.dump(summary, f)
with open("out.json", "r") as f:
    back = json.load(f)
back["n_pass"] = back["n_pass"] + 1
print(back["n_pass"], summary["n_pass"])
```

A. `4 3`
B. `4 4`
✓ **C.** `5 4`
D. `5 5`

Key: C. The scores of at least 65 are 72, 91, 65 and 85, so `n_pass` is 4 (65 counts because of `>=`). `json.load` builds a brand-new dict, so adding 1 to `back["n_pass"]` makes it 5 without touching `summary`, which stays 4. *Tempting wrong answer:* D treats `back` and `summary` as the same object.

24. What does this code print?

```
trials = pd.DataFrame({"subject": ["s1", "s2", "s1", "s2"],
                       "rt":      [500, 900, 700, 400]})
conn = sqlite3.connect("lab.db")
trials.to_sql("trials", conn, index=False, if_exists="replace")
q = """SELECT subject, AVG(rt) AS m FROM trials
       GROUP BY subject ORDER BY subject"""
res = pd.read_sql_query(q, conn)
conn.close()
m = np.array(res["m"])
print(res.shape, (m > 620).sum(), res.loc[1, "subject"])
```

A. `(4, 2) 2 s2`
B. `(2, 2) 1 s1`
C. `(2, 2) 650.0 s2`
✓ **D.** `(2, 2) 1 s2`

Key: D. `GROUP BY subject` returns one row per subject (s1: 600.0, s2: 650.0), so `res.shape` is `(2, 2)`. Only 650.0 is above 620, so the mask sums to 1. Label 1 is the second row, `s2`. *Tempting wrong answer:* B takes label 1 as the first row. Labels start at 0.

25. What does this code print?

```
orders = pd.DataFrame({"cid": [1, 2, 3, 4],
                       "amt": [50, 20, 70, 40]})
cust = pd.DataFrame({"cid":  [1, 2, 3, 5],
                     "city": ["NYC", "LA", "NYC", "SF"]})
m = pd.merge(orders, cust, on="cid", how="left")
m["city"] = m["city"].fillna("unknown")
by_city = m.groupby("city")["amt"].sum().to_dict()
print(by_city["NYC"], by_city["unknown"], len(by_city))
```

A. `120 40 4`
✓ **B.** `120 40 3`
C. `2 1 3`
D. `KeyError: 'unknown'`

Key: B. The left merge keeps all four orders. `cid` 4 has no city, so `fillna("unknown")` labels it `unknown`. Adding up `amt` by city gives NYC $50 + 70 = 120$, LA 20 and unknown 40, which is three entries. *Tempting wrong answer:* D assumes the unmatched order was dropped. That happens with an inner join, not a left join.

Part B: Reading comprehension

One script, 25 points. Read the background, study the data and the script, then answer the questions that follow. Each question says which version of the code it is about. You never need to write code. Describe fixes in words and cite line numbers.

B1. [25 pts] Academic research: a teacher-efficacy survey

Setting. Maya is a research assistant in a university lab that evaluates a coaching program for new teachers. Ten teachers took part in a small pilot: five received coaching and five did not. Each teacher filled out a survey on *classroom-management efficacy*, and one teacher accidentally submitted the form twice. The survey has four items, each answered from 1 (strongly disagree) to 5 (strongly agree). Items `q1`, `q2` and `q4` are worded positively. Item `q3` ("My class is often out of control") is worded negatively. Maya wrote the script `efficacy_pilot.py` (next page) to clean the data and compare the two groups. Her principal investigator (PI) gave her these rules:

R1. Remove duplicate submissions (identical rows) before any analysis.

R2. Reverse-code `q3` so that $1 \leftrightarrow 5$, $2 \leftrightarrow 4$, and 3 stays 3. After that, a higher number always means higher efficacy.

R3. A teacher's *scale score* is the mean of the four items (with `q3` reversed). A teacher who skipped any of the four items gets no scale score and is left out of the group means.

R4. Attach each teacher's school district from `schools.csv`. A teacher whose school is not listed in that file must stay in the data, with the district left blank.

R5. Compare the two group means and save a short summary as JSON for the PI.

The data. The two input files, exactly as pandas displays them after `pd.read_csv` (`NaN` means a blank cell in the file):

`survey_raw.csv`

	resp_id	group	school_id	q1	q2	q3	q4
0	101	coaching	S1	4	5.0	2	4.0
1	102	coaching	S1	5	4.0	1	5.0
2	103	coaching	S2	3	NaN	3	4.0
3	104	coaching	S2	4	4.0	2	3.0
4	104	coaching	S2	4	4.0	2	3.0
5	105	coaching	S3	4	3.0	3	3.0
6	201	comparison	S3	4	3.0	3	4.0
7	202	comparison	S2	2	3.0	4	NaN
8	203	comparison	S1	4	3.0	3	3.0
9	204	comparison	S2	3	4.0	3	3.0
10	205	comparison	S3	3	3.0	4	4.0

`schools.csv`

	school	district
0	S1	North
1	S2	South

Maya's script: `efficacy_pilot.py`

```python
# efficacy_pilot.py: clean the pilot survey and compare the two groups
import json
import os
import pandas as pd

raw = pd.read_csv("survey_raw.csv")
schools = pd.read_csv("schools.csv")
print("rows read:", raw.shape[0])

# Step 1: remove duplicate submissions (identical rows)
df = raw.drop_duplicates().copy()
print("rows after dedupe:", df.shape[0])

# Step 2: how many answers are missing for each item?
items = ["q1", "q2", "q3", "q4"]
print(df[items].isna().sum())

# Step 3: reverse-code q3 (negatively worded; the scale runs from 1 to 5)
df["q3_r"] = 5 - df["q3"]

# Step 4: scale score = mean of the four items, using the reversed q3
df["scale"] = (df["q1"] + df["q2"] + df["q3_r"] + df["q4"]) / 4

# Step 5: attach each teacher's district from the school lookup table
df = pd.merge(df, schools, on="school_id", how="left")
print("rows after merge:", df.shape[0])

# Step 6: compare the two groups on the scale score
summary = df.groupby("group")["scale"].agg(["count", "mean"]).round(2)
print(summary)

# Step 7: difference in group means (coaching minus comparison)
diff = summary.loc["coaching", "mean"] - summary.loc["comparison", "mean"]
print("difference:", round(diff, 2))

# Step 8: save the cleaned table and a short summary for the PI
df.to_csv("survey_clean.csv", index=False)
result = {
    "n_scored": int(summary["count"].sum()),
    "mean_coaching": float(summary.loc["coaching", "mean"]),
    "mean_comparison": float(summary.loc["comparison", "mean"]),
}
with open("summary.json", "w") as f:
    json.dump(result, f)
print("saved:", os.path.exists("summary.json"))
```

About the script. It contains exactly one bug and one PI's rule violation. The bug stops the script with an error message. The rule violation does not stop anything, but it makes the numbers wrong. Each question below says which version of the code it is about.

(a) **[4 pts]** *Code version: Assume the problems in (c) and (d) have both been fixed.* Explain what line 30 prints (it is `print(summary)`). You do not need to calculate any numbers. Just describe in words what it returns, for example what each row and each column stands for. You may also sketch the table if you think that is easier.

Key:

Answer. Line 30 prints the table `summary`. It has one row for each group (`coaching` and `comparison`, shown as the row labels) and two columns. `count` is the number of teachers in that group who have a scale score, and `mean` is the average of those scale scores, rounded to 2 decimals. A teacher who skipped an item has no scale score, so that teacher is not counted. (For reference, with both problems fixed the table shows count 4 and mean 4.00 for coaching, and count 4 and mean 3.25 for comparison.)

(b) **[4 pts]** *Code version: Original script (this answer does not depend on the bug or the rule violation).* In plain English, what do Steps 3 and 4 (lines 18–22) do, and why must Step 3 happen before Step 4?

Key:

Answer. Step 3 reverse-codes `q3`. That item is worded negatively, so agreeing strongly means *low* efficacy. Subtracting it from a constant flips the direction, so that, like the other three items, a higher number means higher efficacy. Step 4 adds up the four items for each teacher and divides by 4, which gives one scale score per teacher. Step 3 has to come first. Otherwise the average would mix items that point in opposite directions, and a teacher who strongly agrees that the class is out of control would get a *higher* score for it.

(c) **[5 pts]** *Code version: Original script.* Maya runs the script. It prints its first three results and then stops with the message `KeyError: 'school_id'`. (i) Which line raises the error? (ii) Explain the cause, using the two data tables. (iii) Describe in words how to fix it.

Key:

Answer. (i) Line 25, the `pd.merge` call. (ii) `schools.csv` names its key column `school`, but the merge asks for a column called `school_id` in both tables. `schools` has no such column, so pandas raises `KeyError: 'school_id'`. (iii) Make the key names match. For example, rename `school` to `school_id` in the `schools` table before the merge, or change the header of `schools.csv`.

(d) **[5 pts]** *Code version: Assume the error in (c) has been fixed, so the script runs to the end.* Even so, one line breaks one of the PI's rules, and Python does not give any error message. (i) Which line is it, and which rule does it break? (ii) Explain the mistake. (iii) Explain how to revise that line.

Key:

Answer. (i) Line 19, which breaks R2. (ii) `5 - df["q3"]` turns 1, 2, 3, 4, 5 into 4, 3, 2, 1, 0, but R2 needs 5, 4, 3, 2, 1. A neutral 3 should stay 3, and here it becomes 2. Every reversed score ends up one point too low, so every scale score is 0.25 too low. (iii) Subtract from 6 instead of 5: `df["q3_r"] = 6 - df["q3"]`. The 6 is the highest possible answer plus 1, so 1 becomes 5, 2 becomes 4, 3 stays 3, 4 becomes 2 and 5 becomes 1.

(e) **[3 pts]** *Code version: Assume the problems in (c) and (d) have both been fixed.* Suppose line 25 used `how="inner"` instead of `how="left"`. What would line 26 print? Write the number, and explain in a sentence or two how you got it. (Hint: which schools are listed in `schools.csv`?)

Line 26 would print `rows after merge:` **7**

Key:

Answer. `schools.csv` lists only S1 and S2. An inner join keeps only the rows that have a match in both tables, so the teachers from S3 are dropped: 105 in the coaching group, and 201 and 205 in the comparison group. Line 26 prints `rows after merge: 7`, which is 10 minus 3.

(f) **[4 pts]** *Code version: Assume the problems in (c) and (d) have both been fixed.* Rule R3 says that a teacher who skipped any of the four items gets no scale score and is left out of the group means. Does the script follow this rule? Answer Yes or No, then explain why. Name the line(s) responsible and point to evidence in the data tables.

Key:

Answer. Yes. Line 22 adds up the four columns, and a blank (NaN) plus anything is NaN, so teacher 103 (blank `q2`) and teacher 202 (blank `q4`) get a NaN scale score. The `count` and `mean` in line 29 skip NaN, so these two teachers are neither counted nor averaged. Evidence: after de-duplication (line 11) each group has 5 teachers, but only 4 of them have a scale score, so line 30 shows a count of 4 in each group.